

Advanced Machine Learning

Optimizers in Brief

Lukas Galke Poech

Spring 2026

Problems with optimizing deep nets

- Non-convex: We can end-up in local minima
- Inexact: Our gradient is not exact but only an estimate
- Poor correspondence between local and global structure (cliffs etc)
- The starting point matters
- The learning rate matters

Now, why are there different optimizers?

- Different optimizers try to avoid or minder the aforementioned problems
- They differentiate in the way they are doing it, often the differences are rather subtle
 - SGD
 - RMSProp
 - Adagrad
 - Adadelta
 - Adam
 - Adamax
 - Nadam
 - AdamW
 - ...

Tricks they use: Decreasing Learning Rate

- Batch* gradient descent (i.e. using all samples) can use a fixed learning rate
 - Since true gradient becomes small and then 0
- SGD has a source of error
 - Random sampling of m training samples
 - Sufficient condition for SGD convergence
$$\sum \varepsilon = \infty; \sum \varepsilon^2 < \infty$$
 - Common to decay learning rate linearly until iteration
$$\tau: \varepsilon_k = (1 - \alpha)\varepsilon_0 + \alpha\varepsilon_\tau \text{ with } \alpha = \frac{k}{\tau}.$$
 - After iteration τ , it is common to leave ε constant
- *This is unfortunately a very confusing terminology: Methods which use the entire dataset are called Batch-methods. Do not confuse with the term "minibatch" from SDG.

Tricks they use: Momentum

- Momentum helps accelerate Gradient Descent(GD) when we have
 - Surfaces that curve more steeply in one direction than in another direction
 - Facing high curvature
 - Small but consistent gradients
 - Noisy gradients
- Dampens oscillation
- Algorithm accumulates moving average of past gradients and move in that direction

Tricks they use: Momentum

- Introduce variable v , or velocity
 - It is the direction and speed at which parameters move through parameter space
 - Momentum in physics is mass times velocity
 - The momentum algorithm assumes unit mass
 - A hyperparameter $\alpha \in [0,1)$ determines exponential decay

$$v \leftarrow \alpha v - \varepsilon \nabla_{\theta} \left(\frac{1}{m} \sum_{i=1}^m L(f(x^{(i)}; \theta), y^{(i)}) \right)$$

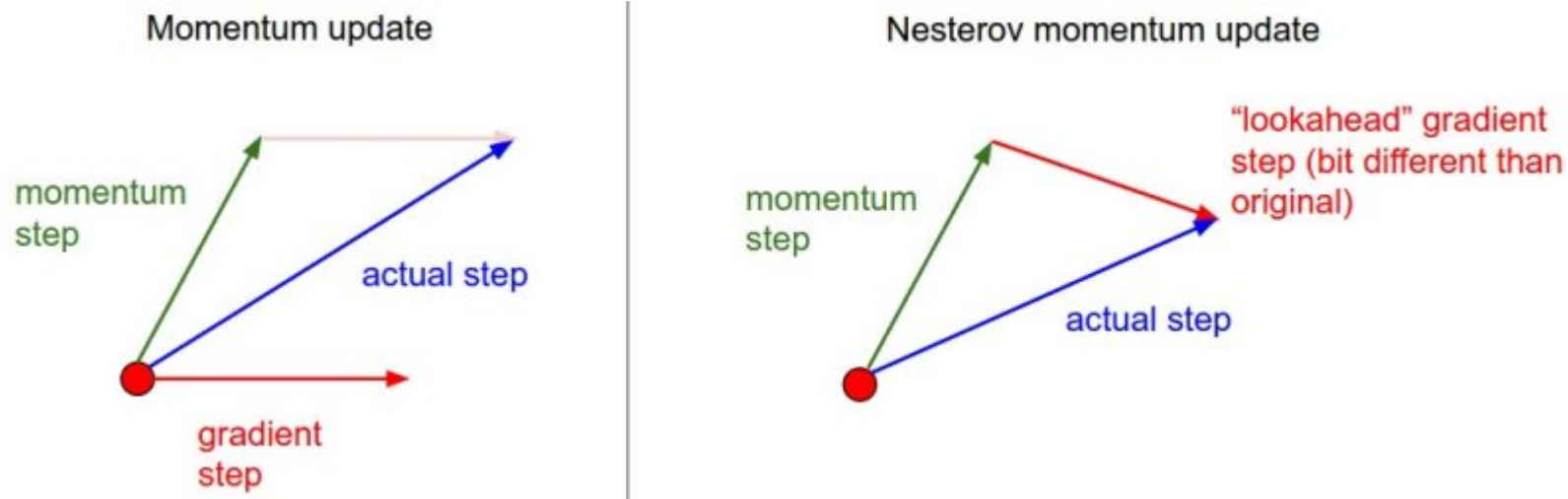
$$\theta \leftarrow \theta + v$$

- The larger α is relative to ε , the more previous gradients affect the current direction

Tricks they use: Nesterov Accelerated Gradient

- Nesterov acceleration optimization is like a ball rolling down the hill but knows exactly when to slow down before the gradient of the hill increases again.
- We calculate the gradient not with respect to the current step but with respect to the future step.
- We evaluate the gradient of the look-ahead and based on the importance then update the weights.

Tricks they use: Nesterov Accelerated Gradient



Tricks they use: Adaptive Learning Rate

- We have discussed how important the learning rate is
 - Set too small, we train very slow
 - Set too large, we might wildly jump above minima
- General Idea of adaptive learning:
 - Modify the learning rate **for individual parameters**
 - Based on their history
 - The different optimizers mainly distinguish each other how they adapt the learning rate.
- This is where the "ADA" comes from in most optimizers

Back to the Optimizers

- **SGD**
- RMSProp
- Adagrad
- Adadelta
- **Adam**
- Adamax
- Nadam
- **AdamW**

Stochastic Gradient Descent

- Can be enhanced by momentum
- Can use Nesterov look-ahead
- Can be combined with learning rate schedulers
- But not adaptive

input : γ (lr), θ_0 (params), $f(\theta)$ (objective), λ (weight decay),
 μ (momentum), τ (dampening), *nesterov*, *maximize*

```
for  $t = 1$  to ... do
   $g_t \leftarrow \nabla_{\theta} f_t(\theta_{t-1})$ 
  if  $\lambda \neq 0$ 
     $g_t \leftarrow g_t + \lambda \theta_{t-1}$ 
  if  $\mu \neq 0$ 
    if  $t > 1$ 
       $\mathbf{b}_t \leftarrow \mu \mathbf{b}_{t-1} + (1 - \tau) g_t$ 
    else
       $\mathbf{b}_t \leftarrow g_t$ 
    if nesterov
       $g_t \leftarrow g_t + \mu \mathbf{b}_t$ 
    else
       $g_t \leftarrow \mathbf{b}_t$ 
  if maximize
     $\theta_t \leftarrow \theta_{t-1} + \gamma g_t$ 
  else
     $\theta_t \leftarrow \theta_{t-1} - \gamma g_t$ 

return  $\theta_t$ 
```

Adam

- Adaptive Moment Estimation
- Next to the exp. dec. avg. of past gradients m_t Adam also keeps an exp. dec. avg. of **squared** past gradients v_t
- Corresponding to estimated mean and ~variance of the gradient
- Larger steps when gradients are small, smaller steps when gradients are large
- Whereas momentum can be seen as a ball running down a slope, Adam behaves like a heavy ball with friction, which thus prefers flat minima in the error surface.

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$
$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

Bias corrected momentums

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t} \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$
$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t$$

AdamW

- For SGD, L2 regularization is equivalent to weight decay
- For Adam, L2 regularization is not equivalent to weight decay
- It can be reconciled: AdamW
- AdamW removes the weight decay term from the initial gradient, but instead adds it when 1st and 2nd moment are consolidated into the parameter update.

Algorithm 1 SGD with L₂ regularization and SGD with decoupled weight decay (SGDW), both with momentum

```

1: given initial learning rate  $\alpha \in \mathbb{R}$ , momentum factor  $\beta_1 \in \mathbb{R}$ , weight decay/L2 regularization factor  $\lambda \in \mathbb{R}$ 
2: initialize time step  $t \leftarrow 0$ , parameter vector  $\theta_{t=0} \in \mathbb{R}^n$ , first moment vector  $m_{t=0} \leftarrow \mathbf{0}$ , schedule multiplier  $\eta_{t=0} \in \mathbb{R}$ 
3: repeat
4:    $t \leftarrow t + 1$ 
5:    $\nabla f_t(\theta_{t-1}) \leftarrow \text{SelectBatch}(\theta_{t-1})$  ▷ select batch and return the corresponding gradient
6:    $g_t \leftarrow \nabla f_t(\theta_{t-1}) + \lambda \theta_{t-1}$ 
7:    $\eta_t \leftarrow \text{SetScheduleMultiplier}(t)$  ▷ can be fixed, decay, be used for warm restarts
8:    $m_t \leftarrow \beta_1 m_{t-1} + \eta_t \alpha g_t$ 
9:    $\theta_t \leftarrow \theta_{t-1} - m_t - \eta_t \lambda \theta_{t-1}$ 
10: until stopping criterion is met
11: return optimized parameters  $\theta_t$ 

```

Algorithm 2 Adam with L₂ regularization and Adam with decoupled weight decay (AdamW)

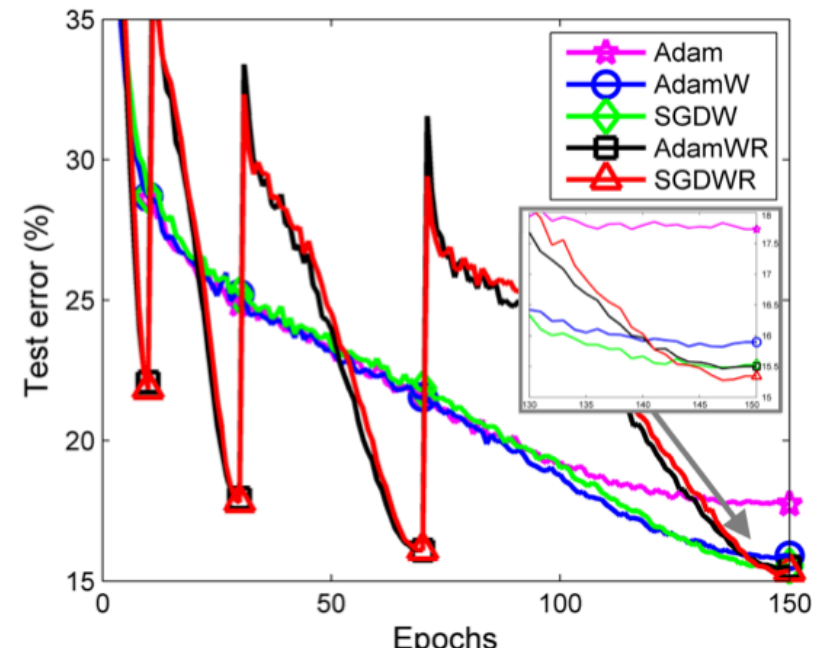
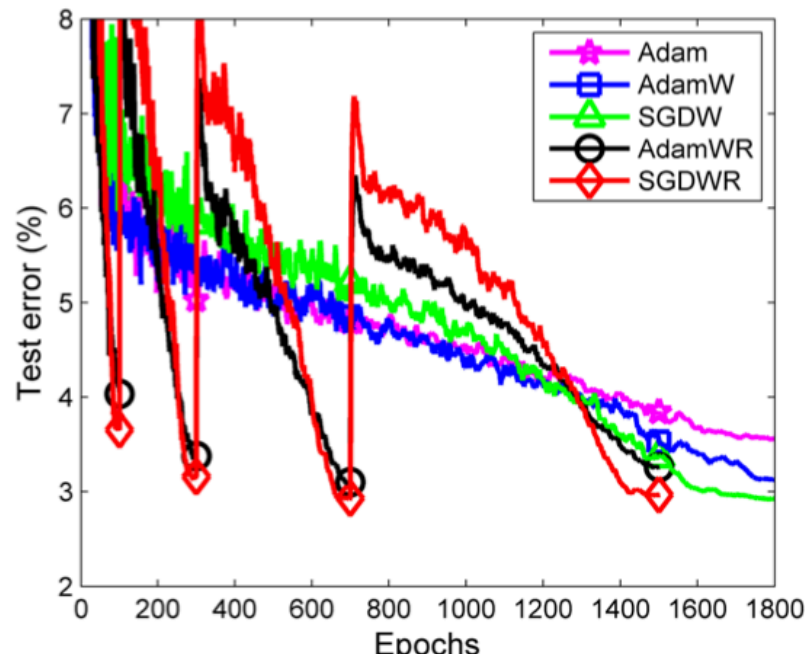
```

1: given  $\alpha = 0.001, \beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-8}, \lambda \in \mathbb{R}$ 
2: initialize time step  $t \leftarrow 0$ , parameter vector  $\theta_{t=0} \in \mathbb{R}^n$ , first moment vector  $m_{t=0} \leftarrow \mathbf{0}$ , second moment vector  $v_{t=0} \leftarrow \mathbf{0}$ , schedule multiplier  $\eta_{t=0} \in \mathbb{R}$ 
3: repeat
4:    $t \leftarrow t + 1$ 
5:    $\nabla f_t(\theta_{t-1}) \leftarrow \text{SelectBatch}(\theta_{t-1})$  ▷ select batch and return the corresponding gradient
6:    $g_t \leftarrow \nabla f_t(\theta_{t-1}) + \lambda \theta_{t-1}$ 
7:    $m_t \leftarrow \beta_1 m_{t-1} + (1 - \beta_1) g_t$  ▷ here and below all operations are element-wise
8:    $v_t \leftarrow \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$ 
9:    $\hat{m}_t \leftarrow m_t / (1 - \beta_1^t)$  ▷  $\beta_1$  is taken to the power of  $t$ 
10:   $\hat{v}_t \leftarrow v_t / (1 - \beta_2^t)$  ▷  $\beta_2$  is taken to the power of  $t$ 
11:   $\eta_t \leftarrow \text{SetScheduleMultiplier}(t)$  ▷ can be fixed, decay, or also be used for warm restarts
12:   $\theta_t \leftarrow \theta_{t-1} - \eta_t \left( \alpha \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon) + \lambda \theta_{t-1} \right)$ 
13: until stopping criterion is met
14: return optimized parameters  $\theta_t$ 

```

AdamW

- TLDR; if you want to use weight decay, always use AdamW instead of Adam



How's my teaching?



<https://www.admonymous.co/lukasgalke>